

The Backtesting Engineer's Field Guide

A complete course on backtesting + a deep tour of this codebase

Who this is for: you, before the demo. By the end you should be able to (1) explain what a backtester is and why it's hard, (2) walk any interviewer through *this* codebase line by line, (3) defend every architectural decision and name its tradeoff, and (4) answer "how would you scale / harden this?" with specifics, not hand-waving. Read it once, slowly. It's built to stick.

Table of contents

Part I — The concepts (be fluent)

- 1 · What a backtest actually is
- 2 · The five components (the universal mental model)
- 3 · The event-driven loop vs the vectorized approach
- 4 · Look-ahead bias — the thing that separates pros from amateurs
- 5 · The other biases & honesty levers (survivorship, slippage, costs)
- 6 · The metrics: Sharpe and Max Drawdown, derived from first principles

Part II — This codebase, component by component

- 7 · Repository map & request lifecycle
- 8 · `engine` — events, the queue, the run loop
- 9 · `data` — Bybit fetch, pagination, caching
- 10 · `strategy` — the pluggable interface, MA & RSI
- 11 · `broker` — fills, fees, slippage
- 12 · `portfolio` — cash, positions, equity curve
- 13 · `metrics` — the math, in code
- 14 · `api` — REST, the worker pool, the job store

Part III — Architecture & decisions (think like a senior)

- 15 · Every decision and its tradeoff, in one table
- 16 · Why a worker pool (and not the alternatives)
- 17 · Why event-driven (and when vectorized wins)

Part IV — Scaling & resilience (the demo-winning section)

- 18 · The performance problem: "seconds" must become "milliseconds"
- 19 · A layered caching & data-storage strategy
- 20 · Scaling the compute: from one box to many

- 21 · Resilience: timeouts, retries, circuit breakers, idempotency
- 22 · Correctness at scale: determinism & reproducibility
- 23 · The production roadmap (what I'd build next, in order)

Part V — The interview

- 24 · Q&A drill — the questions they'll ask, with crisp answers
- 25 · The 60-second whiteboard
- 26 · Glossary

PART I — THE CONCEPTS

1. What a backtest actually is

A **backtest** answers one question: *"If I had run this trading strategy over historical data, how would it have performed — and how much risk would I have taken to get there?"*

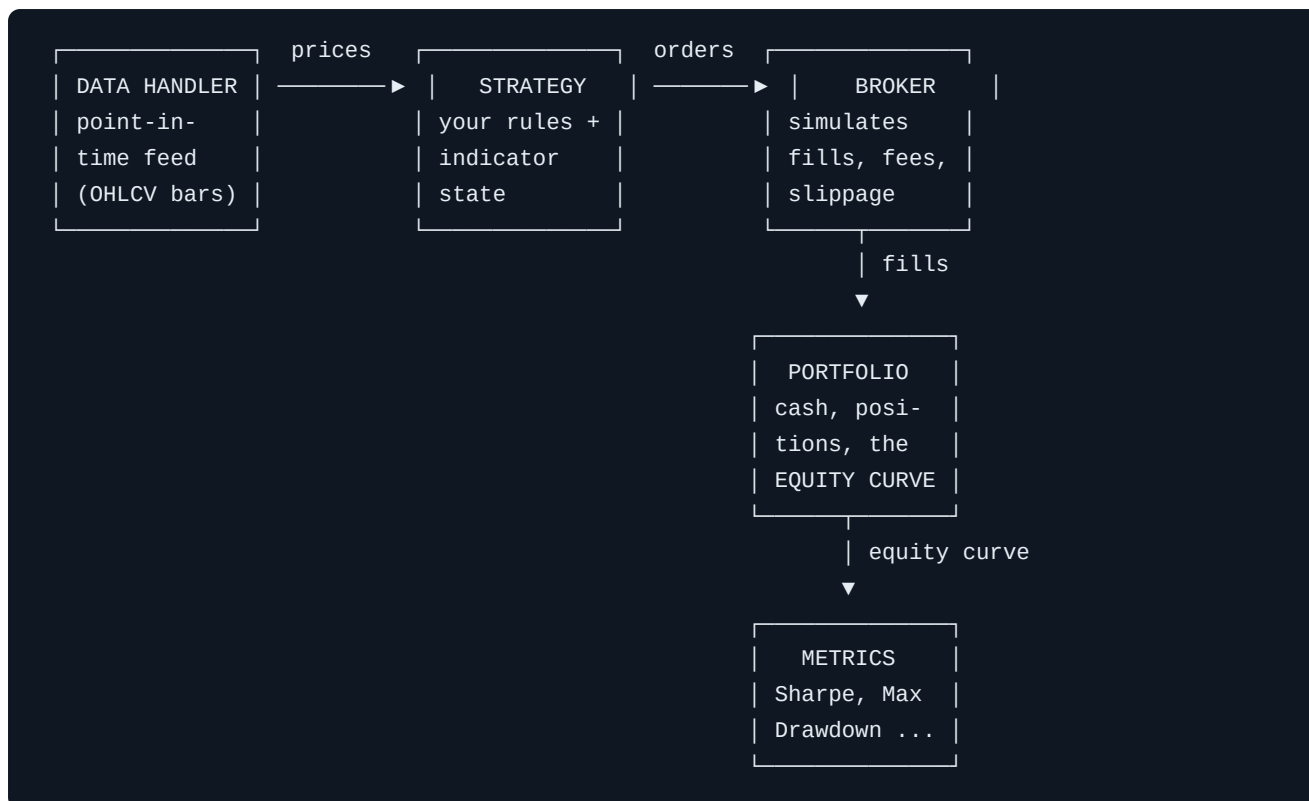
It is a **simulation**. You take a set of rules ("buy when the fast moving average crosses above the slow one"), replay historical prices through those rules **as if you were living through them in real time**, let a simulated broker "fill" the resulting trades with realistic costs, track the account value at every step, and at the end compute performance statistics.

The whole value of a backtester rests on one word: **honesty**. A backtest that lies — by letting the strategy peek at the future, by ignoring trading costs, by testing on a survivor-biased dataset — produces a beautiful equity curve that evaporates the moment real money is deployed. **The engineering challenge of a backtester is not speed or features; it is preventing the simulation from cheating.** Everything senior about this domain flows from that sentence.

Say this in the demo: "A backtester is easy to build and very hard to build *honestly*. My architecture is organized around making dishonesty structurally impossible, not just discouraged."

2. The five components (the universal mental model)

Essentially every serious backtester — open-source or proprietary — decomposes into the same five parts. Know them cold; you can draw this from memory.



1. **Data Handler** — Loads historical bars (OHLCV = Open, High, Low, Close, Volume) and feeds them to the engine **one timestamp at a time, in order**. This is where look-ahead bias is born or prevented.
2. **Strategy** — Receives each new bar, maintains whatever indicator state it needs (a moving average, an RSI value), and emits **orders** (buy/sell intents). It is *pure decision logic*; it doesn't know how orders get filled.
3. **Broker (Execution Handler)** — Takes an order and **models reality**: decides the fill price, applies a fee, applies slippage. In a backtest there is no exchange, so the broker is your model of how the real exchange would have treated your order.
4. **Portfolio** — Tracks **cash**, **open positions**, and the **equity curve** (total account value at every timestamp). The equity curve is the single source of truth from which all metrics are computed.
5. **Metrics / Statistics** — Consume the equity curve (and the trade log) to produce performance numbers: total return, Sharpe ratio, maximum drawdown, win rate, etc.

Why this decomposition matters: each part has one job and a clean boundary, so each can be swapped independently. A new strategy doesn't touch the engine. A new data source doesn't touch the strategy. This is the "support multiple trading systems" requirement, solved by design rather than by special-casing.

3. The event-driven loop vs the vectorized approach

There are two fundamentally different ways to build a backtester. Knowing both — and *why you chose one* — is a senior signal.

3a. Vectorized backtesting

Load the entire price history into arrays (think pandas/NumPy), compute the signal for **every bar at once** with array math, then compute returns in one shot. Example: `signal = (fast_ma > slow_ma)`, `returns = signal.shift(1) * price.pct_change()`.

- **Pros:** extremely fast (it's just matrix math), trivial to write for simple strategies.
- **Cons:** It's a *different program shape* from live trading. It quietly invites look-ahead bias (one wrong `.shift()` and you're trading on the future). It struggles with anything path-dependent: position sizing that depends on current equity, stop-losses, partial fills, portfolio-level risk limits. You cannot reuse the code to trade live.

3b. Event-driven backtesting (what this project uses)

Process the data **one event at a time** through a loop, exactly as a live system would process a stream of incoming market data. Each bar is an event; the strategy reacts; its orders are events; fills are events.

- **Pros: Correct by construction** — it physically processes time in order, so look-ahead is preventable. It handles path-dependent logic naturally (sizing on current cash, stops, etc.). **The same engine shape can later drive live trading** — you just swap the data handler from "historical replay" to "live websocket feed." This is the single biggest architectural advantage.
- **Cons:** slower than vectorized (you're iterating, not doing one big matrix op), and more code.

Decision & defense: "I chose event-driven. A vectorized backtester is faster to run but it's a throwaway script — it can't become a live trading system and it makes look-ahead bias easy to introduce by accident. Event-driven costs raw speed but buys correctness and a path to production. For a *platform*, that's the right trade; for a one-off research notebook, vectorized would be fine."

4. Look-ahead bias — the thing that separates pros from amateurs

Definition: Look-ahead bias is when a strategy uses information it could not possibly have known at the moment it made a decision. It is the #1 reason backtests lie, and it is usually *subtle*.

Three flavors, from obvious to sneaky

1. **Trading on a price you haven't seen yet.** Computing a signal from *today's close* and then "buying at today's close." In reality you don't *know* the close until the bar is over — so you cannot transact at it. The earliest you could act is the **next bar's open**.
2. **Using revised data.** Some datasets get revised after the fact (economic figures, adjusted prices). If your backtest uses the final, revised number at a timestamp where only the preliminary number was actually available, you're cheating.
3. **Indicators that secretly peek.** A centered moving average, or any transform that uses future points to compute a value "at" time T.

How this codebase prevents it — *mechanically, in two layers*

This is the heart of your demo. There are **two independent guarantees**:

Layer 1 — the time-ordered event queue. Events are processed in strict `(timestamp, type)` order. A strategy positioned at time T is, by construction, never handed an event stamped after T. It *cannot* see the future because the data structure won't give it to it.

Layer 2 — next-bar-open execution. A strategy decides on bar T (it's allowed to use T's close, because by the time the decision matters the bar is complete). But the resulting order is **held as pending and filled at the OPEN of bar T+1** — the earliest price a real trader could actually transact at. So even within "the present," the strategy can't trade at the very price it used to decide.

The crucial nuance to articulate: "The event queue stops you seeing *future bars*. But there's a subtler trap — trading at the *current* bar's close, the same close you computed your signal from. My engine closes that hole too, by filling at the next bar's open. The fix actually made my results *more believable*, not just more correct — removing that free edge turned an over-optimistic curve into one a real trader could've achieved."

And it's proven, not asserted. `internal/engine/engine_test.go :: TestNoLookAhead` builds a strategy that buys on bar 0, then asserts the fill price equals **bar 1's open**, not bar 0's close. If anyone ever reintroduces same-bar fills, that test fails. *Showing this test is the strongest possible answer to "how do you know there's no look-ahead?"*

5. The other biases & honesty levers

A senior candidate names these unprompted; it signals you've thought about failure modes.

- **Survivorship bias.** If your universe only contains assets that still exist today, you've silently deleted everything that went bankrupt or got delisted — making every strategy look better than reality. The fix is a **point-in-time** dataset that includes dead assets. (*Less acute for BTC/ETH, which haven't delisted, but you must know the concept.*)
- **Transaction costs.** Every real trade pays a fee. A backtest with zero fees is a fantasy. This engine charges a fee on every fill (default 6 bps).
- **Slippage.** Your order rarely fills at the exact price you wanted; the market moves against you a little. This engine applies **adverse** slippage (buys fill slightly higher, sells slightly lower) — the conservative assumption.
- **Liquidity / market impact.** A large order moves the price. Not modeled here (we assume our size is small relative to BTC/ETH volume), but you should name it as a known simplification.
- **Over-fitting / curve-fitting.** Tuning parameters until the backtest looks great on *this* history, then watching it fail live. The defense is out-of-sample testing, walk-forward analysis, and skepticism of any result that required heavy tuning. (Our configurable params make this easy to *show*: small changes swing the result a lot — that's the lesson, not a bug.)

One-liner: "The honest levers are costs, slippage, point-in-time data, and out-of-sample discipline. A backtester that always prints green is the one you shouldn't trust."

6. The metrics, from first principles

Both required metrics are computed in `internal/metrics/metrics.go` from the **equity curve** — the list of `(timestamp, account_value)` points.

6a. Total return

```
total_return = (final_equity - initial_equity) / initial_equity
```

Simple, but it tells you nothing about *risk* or the *path*. Two strategies can have the same return; one rode a smooth line up, the other survived a 60% crash on the way. That's what the next two metrics capture.

6b. Sharpe ratio — return per unit of risk

Intuition: how much excess return did you earn for each unit of volatility you endured? Higher is better. It penalizes a wild ride.

Formula (as implemented):

```
periodic_return_t = (equity_t - equity_{t-1}) / equity_{t-1}

mean(periodic_returns) - risk_free_rate
Sharpe = ————— × √(periods_per_year)
          stddev(periodic_returns)
```

- We assume **risk-free rate = 0** (fine for a short crypto backtest).
- **stddev** is the *sample* standard deviation (divide by $n-1$).
- **Annualization** is the subtle part. We compute returns *per bar*, so to make Sharpe comparable across timeframes we multiply by `√(periods_per_year)`. And `periods_per_year` differs per timeframe:
 - 1m → 525,600 bars/year (`365 × 24 × 60`)
 - 5m → 105,120
 - 15m → 35,040

This is why minute-bar Sharpes have large magnitudes (you're multiplying by $\sqrt{525,600} \approx 725$). **The sign and the ordering between runs are what matter** — not whether the absolute number "looks normal." Be ready to explain this; it's a common "gotcha" question.

- **Rough scale:** >1 is decent, >2 is good, >3 is excellent (for properly annualized, realistic strategies).

6c. Maximum drawdown — the worst pain

Intuition: the largest peak-to-trough drop in account value over the run. It answers "what's the deepest hole this strategy put me in?" — the thing that makes real people panic-sell.

Algorithm (as implemented): walk the equity curve, track the running maximum (`peak`); at each point the drawdown is `(peak - equity) / peak` ; the **max drawdown** is the worst such value.

```
peak = -∞
worst = 0
for each point p in equity_curve:
    peak = max(peak, p.equity)
    worst = max(worst, (peak - p.equity) / peak)
```

Reported as a fraction (0.25 = a 25% drop). The frontend's **drawdown sub-chart** is exactly this quantity plotted over time (the "underwater" curve).

Both metrics are unit-tested (`metrics_test.go`): a known curve `100→120→90→110` must yield exactly 25% max drawdown; a monotonic curve yields 0; constant returns yield Sharpe 0 (no volatility info). Showing these tests proves the math is pinned down.

PART II — THIS CODEBASE, COMPONENT BY COMPONENT

7. Repository map & request lifecycle

/cmd/api/main.go	→ process entry: builds the worker pool + server, runs it
/internal/engine/	→ THE CORE: event types, the queue, the run loop, interfaces
event.go	→ Bar, Order, Fill, Event, EventType, Side
queue.go	→ EventQueue (a min-heap ordered by time, then type)
engine.go	→ the component interfaces + Run() (the event loop)
engine_test.go	→ TestNoLookAhead, TestOrderOnLastBarIsDropped
/internal/data/	→ Data Handler: Bybit fetch + disk cache
bybit.go	→ FetchKlines (parallel paginated fetch + retry)
handler.go	→ BarFeed (streams bars) + Load (cache-or-fetch)
/internal/strategy/	→ Strategies (pluggable)
strategy.go	→ the Indicating interface + helpers
ma_crossover.go	→ moving-average crossover + indicator series
rsi.go	→ RSI mean-reversion + indicator series
/internal/broker/broker.go	→ Broker: next-open fills, fee + adverse slippage
/internal/portfolio/	→ Portfolio: cash, position, equity curve, trade log
/internal/metrics/	→ Sharpe, Max Drawdown, total return (+ tests)
/internal/api/	→ REST + concurrency
server.go	→ Gin routes & handlers (/api/options, /backtests)
runner.go	→ validates a request, wires components, runs one backtest
pool.go	→ the worker pool + in-memory job store
/web/	→ single-page UI (index.html, app.js, style.css)

The life of one backtest request (trace this end to end)

1. Browser POST `/api/backtests {symbol, interval, strategy, days, params}`
2. `server.go` `handleSubmit` → `validate()` → `pool.Submit(req)`
3. `pool.Submit` assigns a UUID, stores a `Job{status:queued}`, pushes the `id` onto the jobs channel. Returns `202 {id}` immediately. (Async – the HTTP call does NOT block on the backtest.)
4. A worker goroutine pulls the `id`, sets `status:running`, calls `runBacktest()`.
5. `runBacktest`:
 - a. snaps the time window to the bar boundary (for cache stability)
 - b. `data.Load()`: cache hit → read disk; miss → `FetchKlines()` from Bybit, then write cache
 - c. builds the chosen Strategy with the user's params
 - d. builds the Broker (fee + slippage) and Portfolio (initial cash)
 - e. `engine.New(...).Run()` ← the event loop replays every bar
 - f. `metrics.Compute(equity curve)`
 - g. extracts price series + indicator series for the charts
 - h. returns a `BacktestResult`
6. The worker stores the result, sets `status:done`.
7. Browser has been polling GET `/api/backtests/:id` every 150ms; it now sees `status:done` and renders metrics + charts.

Keep that 7-step trace in your head; you can narrate the whole system from it.

8. `engine` — events, the queue, the run loop

This is the package to know best. If you understand `engine`, you understand the backtester.

8a. The data types (`event.go`)

- **Bar** — one OHLCV candle: `Time, Open, High, Low, Close, Volume`. The atom of historical data.
- **Order** — a strategy's *intent*: `Time, Side` (Buy/Sell), `Qty`. Market orders only in this prototype.
- **Fill** — an *executed* order: `Time, Side, Qty, Price, Fee`. The price is post-slippage; the fee is what the broker charged.
- **Event** — the unit that flows through the queue. It carries a `Type` (`MarketEvent` | `OrderEvent` | `FillEvent`), a `Time`, and whichever of `Bar/Order/Fill` is relevant.

8b. The queue (`queue.go`) — why a heap, why two-key ordering

`EventQueue` is a **min-heap** (Go's `container/heap`) ordered by: 1. **timestamp** ascending (earliest event first — this is the time ordering), then 2. **event type** ascending (`MarketEvent=0 < OrderEvent=1 < FillEvent=2`) as the tie-breaker.

Why the type tie-breaker matters: within the *same* timestamp, you want the causal chain to process in the right order — a market event (a bar arrives) before the orders it spawns, before the fills those orders produce. The type ordering guarantees that deterministically. Without it, same-timestamp events could process in arbitrary heap order and results would be non-deterministic.

A heap gives $O(\log n)$ push/pop and always yields the earliest event. That's exactly the abstraction a time-ordered simulation needs.

8c. The interfaces (`engine.go`) — the extensibility story

The engine depends **only** on four interfaces, never on concrete types:

```
type DataHandler interface { Next() (Bar, bool) }
type Strategy      interface { Name() string; OnBar(bar Bar, position, cash float64) []Order }
type Broker        interface { Execute(order Order, bar Bar) Fill }
type Portfolio     interface { OnFill(Fill); MarkToMarket(Bar); Position() float64; Cash() float64 }
```

This is the **"support multiple trading systems"** requirement, solved: the engine doesn't know whether it's running MA crossover or RSI, fetching from Bybit or reading a CSV. **Adding a new strategy is writing one file** that implements `Strategy` — zero engine changes. That sentence is worth a lot in the interview.

8d. The run loop (`engine.go :: Run`) — read this until it's automatic

The loop is small but every line is deliberate. In plain English:

```
pending = [] // orders decided last bar, waiting to fill

for each bar from the data handler:
    create a fresh event queue for this bar
    push a MarketEvent for this bar

    // FILL LAST BAR'S DECISIONS at THIS bar's open (anti-look-ahead, layer 2)
    for each order in pending:
        push an OrderEvent (stamped at this bar's time)
    pending = []

    // drain this bar's causal chain in (time,type) order
    while queue not empty:
        ev = pop()
        MarketEvent → orders = strategy.OnBar(bar, position, cash)
                        pending += orders // NOT filled now – next bar
        OrderEvent  → fill = broker.Execute(order, bar) // fills at bar.Open
                        push FillEvent
        FillEvent   → portfolio.OnFill(fill)

    portfolio.MarkToMarket(bar) // record equity at this bar's close
```

Three things to be able to defend:

1. **Why `pending` ?** It is the mechanism of next-bar-open execution. Orders born on bar T sit in `pending` and only convert to OrderEvents when bar T+1 starts. This is the look-ahead fix, made structural.

2. **Why mark-to-market at the close?** The equity curve needs exactly one point per bar, recorded *after* that bar's fills are applied, valued at the bar's close. That gives a clean per-bar return series for Sharpe.
 3. **What about an order on the very last bar?** There's no next bar to fill it at, so it's dropped — we refuse to invent a price. `TestOrderOnLastBarIsDropped` pins this behavior.
-

9. `data` — Bybit fetch, pagination, caching

9a. `BarFeed` and the `DataHandler` boundary (`handler.go`)

`BarFeed` is a trivial cursor over a pre-loaded slice: `Next()` returns the next bar or `false`. **All the look-ahead safety reduces to iteration order** — the feed simply hands out bars oldest-first, and the engine never asks for more than the current one. Loading happens up front; the engine only ever sees `Next()`.

`Load(cacheDir, symbol, interval, start, end)` is the cache-or-fetch front door: read the cache file if present; otherwise fetch from Bybit and write the cache. The cache file is keyed by `symbol_intervalm_start_end.json`.

9b. Bybit specifics (`bybit.go`) — the realities of a real API

- Endpoint: `GET /v5/market/kline?category=spot&symbol=...&interval=...&start=...&end=...&limit=1000`.
- Bybit returns at most **1000 bars per request, newest-first**, and as **arrays of strings** (`[startMs, open, high, low, close, volume, turnover]`) which we parse into typed `Bar` s.
- 30 days of 1-minute bars $\approx$ **43,200 bars** $\rightarrow$ $\approx$ 44 pages. **This is the source of the multi-second cold fetch**, and the thing Part IV is about.

9c. Two performance/robustness features already in place

1. **Parallel paginated fetch.** Because we know the full time range and the 1000-bar page size up front, we compute *all* page windows ahead of time and fetch them **concurrently** with a bounded pool (`maxConcurrency = 8`). This turned ~ 44 *sequential* round-trips into a few parallel batches — the single biggest fetch speedup, and what turned a timeout into a reliable ~ 11 s worst case (typical 15m/30d is ~ 1 – 2 s).
2. **Retry with backoff.** Each page request retries up to $4\times$ with 1s/2s/4s backoff (`fetchPageWithRetry`). This is what fixed the original `context deadline exceeded` error — a transient hiccup no longer fails the whole backtest.

After fetching, results are **deduplicated** (by millisecond timestamp) and **sorted oldest-first** before being handed to the engine. Dedup matters because adjacent page windows can overlap at the boundary.

9d. The cache-key subtlety (a real bug we fixed — great story)

Originally the time window was `time.Now()` at request time, so every run had a slightly different `start/end`, the cache key never matched, and reruns re-fetched every time. The fix: **snap to down to**

the current bar boundary (`time.Now().Truncate(interval)`) so repeated runs within the same bar produce an *identical* window → identical cache key → instant rerun (~15ms vs ~3s). This is a good "here's a subtle bug I found and fixed" anecdote — it shows you test your own assumptions.

10. `strategy` — the pluggable interface, MA & RSI

A strategy is **pure decision logic with private indicator state**. It receives a bar plus the current `position` and `cash`, and returns zero or more orders. It never touches fills, fees, or the engine.

10a. MA Crossover (`ma_crossover.go`)

- State: a growing slice of closes, plus the previous fast/slow MA values.
- Rule: compute fast & slow **simple moving averages**; when fast crosses *above* slow → go long (buy with all cash); when fast crosses *below* slow → exit (sell the whole position).
- **Periods are in BARS, not days.** `fast=10, slow=30` means a 10-bar and a 30-bar average. What that is in wall-clock time depends entirely on the timeframe: on 15m bars, $10/30 = 2.5h / 7.5h$. (*This was your exact question — the answer is "bars," and the UI now labels them that way and lets the user set them.*)
- It records the fast & slow series each bar and exposes them via `Indicators()` so the frontend can overlay them on the price chart.

10b. RSI Mean-Reversion (`rsi.go`)

- Computes the **Relative Strength Index** with Wilder's smoothing over `period` bars: seed the average gain/loss over the first `period` changes, then smooth.
- Rule: when `RSI < oversold` (default 30) → buy; when `RSI > overbought` (default 70) → sell. ("Mean reversion": bet that extremes snap back.)
- Exposes the RSI series + the band thresholds, which the frontend draws in a dedicated 0–100 sub-panel.

10c. The `Indicating` interface (`strategy.go`)

```
type Indicating interface { Indicators() map[string][]IndicatorPoint }
```

The runner checks `if ind, ok := strat.(strategy.Indicating); ok { ... }`. So a strategy *may* expose chartable series, but isn't required to — clean optional capability via a small interface and a type assertion.

The extensibility pitch, concretely: "To add, say, a Bollinger-band breakout strategy, I write one file implementing `OnBar`, optionally `Indicators()`, register it in `buildStrategy`, and add its param spec to `/api/options`. The engine, broker, portfolio, metrics, and even most of the frontend are untouched."

11. `broker` — fills, fees, slippage

`Simulated` is the broker. `Execute(order, bar)` returns a `Fill` :

- **Fill price** = `bar.Open` (the next bar's open, because of when the engine calls it) adjusted for **adverse slippage**: buys at `open × (1 + slip)` , sells at `open × (1 - slip)` . Slippage *always* hurts you — the conservative, honest assumption.
- **Fee** = `price × qty × feeRate` (default 6 bps ≈ Bybit taker).

It's deliberately tiny, and deliberately *behind an interface* — the engine only knows `Broker.Execute` . The roadmap version is a more realistic broker: volume-dependent slippage, maker/taker fee tiers, partial fills, limit orders, order rejection when liquidity is thin. **None of that changes the engine.**

Why fills here punish over-trading: every round-trip pays fee + slippage on both legs. A strategy that whipsaws hundreds of times bleeds out — which is *correct*, and visible in the numbers (more trades → worse net result). That's the broker doing its honesty job.

12. `portfolio` — cash, positions, equity curve

The portfolio is the **bookkeeper** and the **source of truth**.

- `OnFill(fill)` : a buy spends `price×qty + fee` of cash and adds `qty` to the position; a sell does the reverse. Every fill is appended to a **trade log**.
- `MarkToMarket(bar)` : records `equity = cash + position × bar.Close` onto the **equity curve** — one point per bar.
- Exposes `Position()` , `Cash()` , `Equity()` , `Trades()` .

The equity curve is everything. Every metric, every chart derives from it. This is why the design routes all value-tracking through one place: if the equity curve is right, the metrics are right.

One honest simplification to be able to name: the strategy sizes a buy as `cash / current_close` , but the fill happens at the *next* open at a slightly different price — so cash can dip marginally negative on a gap. For a long-only demo this is immaterial; the production fix is to size at fill time or hold a small cash buffer. Naming your own simplifications builds trust.

13. `metrics` — the math, in code

Covered in §6 conceptually. In code (`metrics.go`):

- `Compute(equity, numTrades, interval)` builds the **per-bar return series** from the equity curve, then calls `sharpe(...)` and `maxDrawdown(...)` .

- `sharpe` uses sample stddev and annualizes with `barsPerYear[interval]` (525,600 / 105,120 / 35,040 for 1m/5m/15m). Returns 0 when there's no volatility (undefined risk-adjusted return).
- `maxDrawdown` is the running-peak algorithm from §6c.
- Edge cases handled: fewer than 2 equity points → zeroed result; zero previous equity → that return treated as 0.

The two test files are your proof of correctness — keep them in mind as "evidence" you can point an interviewer to.

14. `api` — REST, the worker pool, the job store

14a. Routes (`server.go`)

- `GET /api/options` — advertises the fixed surface (products, timeframes) **and each strategy's tunable params** (key, label, default, min, max). The frontend builds its inputs from this, so adding a param is a backend-only change.
- `POST /api/backtests` — validates, enqueues, returns `202 {id}`. **Async**.
- `GET /api/backtests/:id` — returns the job (and the full result once done).

14b. The worker pool & job store (`pool.go`) — the concurrency story

```
jobs := make(chan string, queueSize) // bounded queue
for i := 0; i < numWorkers; i++ { go worker() } // fixed N workers
```

- `Submit` stores a `Job{queued}` and does a **non-blocking** channel send (`select { case jobs<-id: ... default: return full }`). If the queue is full it returns "full" → the API responds `503`. **This is back-pressure** — the system refuses work gracefully instead of melting down.
- Each `worker` pulls an id, runs the backtest with **fully isolated state** (its own strategy/broker/portfolio instances), and writes the result back under a mutex.
- The store is an in-memory `map[string]*Job` guarded by a `sync.RWMutex`.

Why this design (say it crisply): "I cap concurrency with a fixed worker pool instead of spawning a goroutine per request. If 500 users hit `run` at once, goroutine-per-request launches 500 CPU-bound backtests that thrash the box into the ground. A worker pool runs N at a time and *queues* the rest, so the system degrades gracefully — latency rises, but it stays up. The bounded queue adds back-pressure so we shed load rather than OOM."

PART III — ARCHITECTURE & DECISIONS

15. Every decision and its tradeoff (one table)

Area	Decision	Why	Tradeoff / when it's wrong
Engine model	Event-driven loop	Correct-by-construction; reusable for live trading; handles path-dependent logic	Slower than vectorized; more code. Vectorized wins for quick research sweeps.
Look-ahead	Time-ordered queue + next-open fills	Two independent guarantees; unit-tested	Slightly more conservative fills than "trade at close"; that's the honest cost.
Ordering	Min-heap on (time, type)	Deterministic causal order within a timestamp	Heap overhead vs a plain queue; negligible here.
Extensibility	Strategy/Broker/Data/Portfolio interfaces	"New trading system = one file"; swap data source freely	Indirection; for a 1-strategy tool it'd be over-engineering.
Concurrency	Fixed worker pool + bounded queue	Graceful degradation, back-pressure, isolated runs	Caps throughput; a huge burst queues/sheds. Needs tuning N.
API shape	Async (POST → poll)	Exercises the pool; scales to long jobs; non-blocking HTTP	Polling overhead; a streaming/websocket or webhook is nicer at scale.
Data fetch	Live Bybit + parallel pages + retry	Real integration; resilient to hiccups	External dependency & latency (Part IV).
Caching	On-disk JSON, key snapped to bar	Instant reruns; offline-safe demo	JSON is fat & slow vs columnar; single-node only.
Storage	In-memory job store	Trivial for a prototype	Not durable, not multi-instance. Postgres is the next step.
Metrics	Equity-curve-derived, annualized Sharpe	One source of truth; correct per-timeframe	Big minute-bar Sharpe magnitudes need explaining.
Costs	Fee + adverse slippage	Honest, punishes over-trading	Simplified (flat bps, no market impact).

If you can talk through this table, you can handle the architecture portion of the interview.

16. Why a worker pool (and not the alternatives)

- **Goroutine-per-request:** simplest, but unbounded concurrency → resource exhaustion under load. Rejected.
- **Single worker (serial):** safe but wastes a multi-core box; one slow job blocks everyone. Rejected.
- **Worker pool (chosen):** bounded parallelism = predictable resource use + graceful degradation. The size N is the tuning knob ($\approx$ number of cores for CPU-bound replay).

- **Next step beyond a pool:** an external queue (Redis/NATS/SQS) so jobs survive restarts and many API instances share one work queue (see §20).

17. Why event-driven (and when vectorized wins)

Covered in §3. The crisp version: **event-driven for a platform that may go live and must be honest; vectorized for throwaway research where speed > realism.** Knowing *when each is right* is the senior move — don't dogmatically claim one is always better.

PART IV — SCALING & RESILIENCE (the section they'll grill you on)

Framing for the demo: "The prototype fetches in seconds. For a research tool that's fine. For an *operational* product it must be milliseconds and it must not fall over when Bybit hiccups or 500 users hit run. Here's exactly how I'd get there." Then walk these sub-sections.

18. The performance problem: seconds → milliseconds

Where the seconds go today: the *cold* network fetch from Bybit (tens of paginated HTTP calls). The replay itself (tens of thousands of bars through the loop) is fast — single-digit milliseconds to low hundreds. **So the bottleneck is data I/O, not compute.** That's the key diagnosis to state out loud, because it dictates the fix: *you make data access fast, not the engine faster.*

The principle: never fetch on the hot path. An operational backtester does not call the exchange when a user clicks run. It serves from a local store that was populated *ahead of time*. The exchange call becomes a background ingestion concern, not a request-time concern.

The ladder, fastest at the bottom:

slowest		
	Bybit REST (cold) ~ seconds	← today's cold path
	Local columnar files (Parquet) ~ 10-100 ms	
	Time-series DB (Timescale/ClickHouse) ~ 1-20ms	
	In-process / Redis cache ~ sub-ms-few ms	
fastest		

19. A layered caching & data-storage strategy

1. **Pre-ingestion (the big one).** A background job continuously pulls Bybit klines for the supported symbols/timeframes and stores them locally. By the time a user runs a backtest, the data is *already*

here. Request-time fetches drop to zero for the common case. This single change converts "seconds" into "the speed of your local store."

2. **Replace JSON with a columnar format.** Today's cache is JSON — fat and slow to parse. **Parquet** (or Arrow) is columnar, compressed, and an order of magnitude faster to load; OHLCV is a perfect fit. Easy, high-leverage win.
3. **Use a purpose-built store for bars.** **TimescaleDB** (Postgres extension) or **ClickHouse** are built for exactly this (huge ordered time-series, range scans by symbol+time). They give you indexed `WHERE symbol=... AND time BETWEEN ...` in milliseconds and they scale past one machine's disk.
4. **Hot cache for repeats.** Memoize *results* by `(symbol, interval, range, strategy, params)` in Redis or in-process. Identical re-runs (very common when a user tweaks one knob) return instantly. The bar boundary snapping we already do makes these keys stable.
5. **Cache the bars in memory too.** The raw OHLCV for "BTCUSDT 15m last 30d" is small; keep a hot window resident so the *replay* never even touches disk.

The headline answer: "I'd separate ingestion from execution. A background pipeline keeps a local Parquet/Timescale store warm, so a backtest never calls Bybit on the user's click — it reads pre-staged data in milliseconds. The exchange becomes an ingestion dependency, not a request-time one. Then I add a Redis result cache for repeated runs. That's how seconds become milliseconds."

20. Scaling the compute: from one box to many

- **Vertical first.** The replay is CPU-bound; size N workers to cores and you saturate a box cheaply. Often enough.
- **Externalize the queue.** Swap the in-process channel for **Redis/NATS/SQS**. Now jobs are durable (survive restarts) and *many* API instances + *many* worker instances share one queue. The API and the workers scale independently.
- **Stateless workers + shared stores.** Workers hold no cross-run state, so you scale them horizontally behind the queue. Results and bars live in shared Postgres/Timescale + object storage, not in a worker's memory.
- **Partition the heavy jobs.** A parameter sweep (try 100 MA combos) is embarrassingly parallel — fan out one job per combo across the worker fleet, then aggregate.
- **Cache results** (§19.4) so the cheapest backtest is the one you don't re-run.

Progression to recite: "Single box with a worker pool → externalize the queue for durability and multi-instance → stateless horizontal workers → fan-out for sweeps → result caching. Each step is a small, independent change because the boundaries are already interfaces and a queue."

21. Resilience: timeouts, retries, circuit breakers, idempotency

What "resilient" means concretely, and what's already here vs. next:

- **Timeouts** — every external call must have one. (*Have it: 30s HTTP client.*)
- **Retries with backoff** — absorb transient failures. (*Have it: 4× with 1/2/4s.*) Next: add **jitter** to avoid thundering-herd, and cap total retry time.
- **Circuit breaker** — if Bybit is *down* (not just slow), stop hammering it; fail fast and serve cached data or a clear error. Prevents one dependency's outage from cascading into yours.
- **Rate limiting** — respect Bybit's limits (our `maxConcurrency=8` is a crude version); add a token-bucket limiter on the ingestion side.
- **Graceful degradation** — already present via the bounded queue + 503. Extend with: serve stale-but-cached data when live fetch fails ("last known good").
- **Idempotency** — a retried `POST /backtests` shouldn't run twice. Key the job by a hash of its inputs; a duplicate returns the in-flight/finished job. (*This is exactly the idempotency-key pattern from your Chapa work — a strong thing to bridge to.*)
- **Bulkheads** — isolate ingestion workers from backtest workers so a data outage can't starve the compute pool.
- **Observability** — structured logs, metrics (fetch latency, queue depth, job duration, cache hit rate), tracing. *You can't harden what you can't see.*

22. Correctness at scale: determinism & reproducibility

As you scale, *correctness* gets harder, not easier. Senior candidates raise this:

- **Determinism.** Same inputs must yield the same result on any worker, any time. Our `(time, type)` heap ordering ensures the replay is deterministic. Pin strategy params, fee/slippage config, and the exact data version.
- **Golden-file tests.** Lock a known strategy + a frozen dataset to an expected equity curve / metric set. If a refactor changes the numbers, the test screams. (Your prep doc already lists this — it's the reproducibility guarantee.)
- **Data versioning.** Bars can be corrected by the exchange. Store *which* version a run used so results are reproducible months later (and so you avoid the "revised data" look-ahead trap from §4).
- **Immutability.** Treat a stored backtest run as immutable: inputs, data version, code version (git SHA), and outputs. That's an audit trail a fund or a serious trader will demand.

23. The production roadmap (what I'd build next, in order)

Say it as a sequence — it shows you can prioritize:

1. **Durable storage:** Postgres for runs/trades/metrics; Parquet/Timescale for bars. (Removes the in-memory store and the JSON cache.)
2. **Background ingestion pipeline:** keep the bar store warm → millisecond request-time data.
3. **Result caching (Redis):** instant repeat runs.
4. **Externalized queue:** durable jobs, independent API/worker scaling.
5. **Richer broker & metrics:** limit orders, market impact, partial fills; Sortino, Calmar, win rate, profit factor, exposure.
6. **More strategies + parameter sweeps / walk-forward** (fan-out compute).

7. **Resilience hardening:** circuit breaker, idempotency, rate limiting, observability.

8. **Packaging & deploy:** Docker, CI, golden-file tests in the pipeline.

PART V — THE INTERVIEW

24. Q&A drill (answer out loud before reading the answer)

Q. What is a backtester, in one breath? A simulation that replays historical prices through a strategy, one step at a time, with realistic costs, to measure how it *would* have performed and the risk it took — its whole job is to do that *honestly*.

Q. How do you prevent look-ahead bias? Two layers. A strictly time-ordered event queue means a strategy at time T can never be handed data after T. And orders decided on bar T fill at bar T+1's *open*, not the close they were computed from. It's mechanical and unit-tested (`TestNoLookAhead`), not a matter of discipline.

Q. Why event-driven instead of vectorized? Correctness and reuse. Vectorized is faster but it's a throwaway script that invites look-ahead and can't become a live system. Event-driven processes time in order, handles path-dependent logic, and the same engine can later drive live trading by swapping the data handler.

Q. How do you handle many concurrent backtests? A fixed-size worker pool with a bounded queue. It caps concurrency so the box degrades gracefully under load instead of crashing, sheds load with back-pressure (503) when the queue is full, and runs each backtest with fully isolated state.

Q. The fetch takes seconds — that's not operational. Fix it. The bottleneck is data I/O, not compute. So I move the exchange call *off* the hot path: a background pipeline pre-ingests bars into a fast local store (Parquet/Timescale), so a run reads pre-staged data in milliseconds. Add a Redis result cache for repeated runs. The replay itself is already fast.

Q. How is the Sharpe ratio computed, and why are the numbers so large on 1m? Mean of per-bar returns over their stddev, annualized by $\sqrt{\text{periods per year}}$. Minute bars have 525,600 periods/year, so the $\sqrt{}$ factor (~ 725) inflates the magnitude. The sign and relative ordering are what matter; absolute scale is a function of the timeframe's annualization.

Q. What's max drawdown? The worst peak-to-trough drop in the equity curve. Track the running peak; the drawdown at each point is $(\text{peak} - \text{equity})/\text{peak}$; the max of that over the run. It's the deepest hole the strategy would've put you through.

Q. How do you make results realistic? Model a fee and adverse slippage on every fill, fill at the next open (no free edge), and be honest about simplifications (no market impact, point-in-time data needed for delistable universes).

Q. How would you store results in production? Postgres: a `runs` table, a `trades` log, equity-curve rows, a `metrics` row per run — plus the code SHA and data version for reproducibility. Bars go in a

columnar/time-series store, not JSON.

Q. How do you add a new strategy? Write one file implementing `OnBar` (and optionally `Indicators()`), register it in `buildStrategy`, and declare its params in `/api/options`. Engine, broker, portfolio, and metrics are untouched.

Q. What breaks first under 100× load, and what do you do? The in-memory job store and the single-process queue. I externalize the queue (Redis/SQS) for durability and multi-instance sharing, move results to Postgres, and scale stateless workers horizontally — all small changes because the boundaries are already a queue and interfaces.

Q. How do you know the engine is correct? Determinism via (time,type) ordering, unit tests on the metrics math and the no-look-ahead guarantee, and (next step) golden-file tests pinning a known strategy+dataset to an expected result.

25. The 60-second whiteboard

Draw and narrate:

```
Browser -POST-> API -enqueue-> [bounded queue] -> Worker Pool (N)
                                     | one isolated run
                                Data(Bybit+cache) -> ENGINE: Data->Strategy->Broker->Portfolio
                                     (all via a time-ordered EVENT QUEUE,
                                     fills at NEXT bar's open)
                                     | equity curve
                                     |> Metrics (Sharpe, MaxDD)
                                     |> (prod: Postgres + Redis)
```

Talking track: "Time-ordered queue + next-open fills = no look-ahead, proven by test. Interfaces = new strategy is one file. Worker pool = graceful under load. And to make it operational I pre-ingest data so the hot path is milliseconds, not seconds."

26. Glossary

- **OHLCV** — Open/High/Low/Close/Volume; one row = one bar/candle.
- **Bar / candle** — price summary for one time interval.
- **Equity curve** — account value at every timestamp; source of all metrics.
- **Look-ahead bias** — using information not yet knowable at decision time.
- **Survivorship bias** — testing only on assets that still exist today.
- **Slippage** — difference between expected and actual fill price.
- **Sharpe ratio** — annualized mean return ÷ stddev of returns; return per unit of risk.
- **Max drawdown** — largest peak-to-trough equity drop.
- **Mean reversion** — strategy betting that extremes revert (e.g. RSI).
- **Trend following** — strategy betting moves continue (e.g. MA crossover).
- **Event-driven** — process one event at a time, in time order.
- **Vectorized** — compute over whole arrays at once.

- **Back-pressure** — refusing/queuing new work when at capacity instead of collapsing.
 - **Idempotency** — a repeated request has the same effect as one request.
 - **Circuit breaker** — stop calling a failing dependency to prevent cascades.
 - **Walk-forward** — re-fit/test on rolling out-of-sample windows to fight over-fitting.
-

You built this. You fixed a real look-ahead bug, a real cache bug, and a real timeout — and you can explain why each fix is correct. Walk in calm, lead with correctness, and let the working demo + this understanding do the talking.